HAAR and HOG

Structured Notes

Python Implementation of HAAR

Cascade and HOG
Feature Extraction for Face and Object Detection

Contents

1 Introduction to HAAR **3**
1.1 Why Use HAAR? . . . 3

2 Steps Involved in HAAR **3**
2.1 Step 1: Grayscale Conversion . . . 3
2.2 Step 2: Integral Image Computation . . . 3
2.3 Step 3: HAAR Feature Calculation . . . 3
2.4 Step 4: Weak Classifiers and Stages . . . 4
2.5 Step 5: Strong Classifier . . . 4

3 Summary of the HAAR Algorithm **4**

4 Python Implementation of HAAR **4**
4.1 Libraries Required . . . 4
4.2 Loading the XML Cascade Files . . . 5

5 Structure of a HAAR Cascade XML File **5**
5.1 Top-Level Tags . . . 5
5.2 Weak Classifier Structure . . . 5
5.3 Number of Weak Classifiers per Stage . . . 5
5.4 Defining HAAR Features in XML . . . 6

6 Face and Eye Detection Functions **6**
6.1 Detecting Faces . . . 6
6.2 Detecting Eyes . . . 7
6.3 Reading and Copying the Image . . . 7

7 Combined Face and Eye Detection **7**

8 Detecting Multiple Faces **8**

9 Non-Maximum Suppression for Overlapping Boxes **8**

10 Introduction to HOG **8**

11 Steps Involved in HOG **9**
11.1 Step 1: Compute Gradients . . . 9
11.2 Step 2: Divide Image Into Cells . . . 9
11.3 Step 3: Histogram of Gradient Directions . . . 9
11.4 Step 4: Block Normalization . . . 9
11.5 Step 5: Combine Histograms . . . 9

12 Why Use HOG and Its Limitations **9**
12.1 Advantages . . . 9
12.2 Limitations . . . 9

13 Python Implementation of HOG **9**
13.1 Libraries Required . . . 9
13.2 HOG on an Astronaut Image . . . 10
13.3 HOG on Other Images (e.g., Cup and Saucer) . . . 10

14 Varying HOG Parameters **10**

15 Practical Notes on HAAR vs HOG **10**

15.1 When to Use HAAR 10
15.2 When to Use HOG 11
15.3 Limitations of Classical Algorithms 11
15.4 Combining with Deep Learning 11

16 Parameter Tuning for Face Detection **11**

17 Summary: Key Points **11**

1 Introduction to HAAR

HAAR Features. HAAR features are simple rectangular patterns which are used to detect objects in an image. These patterns contain black and white regions, and the algorithm is able to calculate the difference in pixel intensities between these regions.

For example, when HAAR features are applied on a face, eyes are usually darker than cheeks; HAAR captures this contrast. The nose is brighter than eyes, which is another HAAR feature. Instead of using any other means, HAAR just calculates the intensity difference in order to understand the image and find the object or face.

1.1 Why Use HAAR?

- HAAR is faster. It works on something called an **integral image**.
- It is used for real-time detection.
- Without any other learning or pattern, it simply learns the pixel intensity differences.
- It focuses on important patterns like edges, lines, and textures in an image.
- By doing so, it reduces the computation cost and improves the efficiency.
- It works with simple classifiers like AdaBoost.
- It is used for face detection, eye detection, and object detection overall.

2 Steps Involved in HAAR

2.1 Step 1: Grayscale Conversion

HAAR first converts the RGB image into a grayscale image.

- **RGB image:** has three channels — red, green, and blue.
- **Grayscale image:** has just a single channel.

By doing so, it is easier to capture the features in grayscale because HAAR depends on intensity, not color. This reduces the computational complexity by going from three channels to one channel. This conversion does not affect any kind of structural patterns in an image — the patterns, structure, or information in an image remain the same. HAAR depends on intensity difference; it doesn't mean color information.

2.2 Step 2: Integral Image Computation

Integral Image. For each pixel in a grayscale image, we calculate the cumulative sum by summing up the pixel intensities of the pixels lying above, to the left, and diagonal to the pixel.

2.3 Step 3: HAAR Feature Calculation

After calculating the integral image of the original image, we calculate the HAAR features. We place the HAAR filter (HAAR feature) on an image and then calculate the sum of white pixels and the sum of black pixels.

Values used in HAAR feature computation:

$$\text{white portion} \to +1, \qquad \text{black portion} \to -1$$

For the window on which we slide this HAAR feature, we calculate the difference between the sum of white pixels and the sum of black pixels. Equivalently, when we place the HAAR feature on a window in an image, we calculate the correlation — this gives us the HAAR feature.

2.4 Step 4: Weak Classifiers and Stages

This HAAR feature value is compared with a threshold. The value that we get is called the value from a **weak classifier**. Many such weak classifiers are present in one stage.

> **Multi-stage Cascade.** In HAAR, we can have multiple stages. We divide the image into windows; for each window we use multiple stages. In each stage we calculate HAAR features, and each stage can have as many weak classifiers as needed (sometimes nine, sometimes ten). Each weak classifier calculates a HAAR feature. If the test is passed by one stage, the window goes to the next stage; otherwise we stop there.

2.5 Step 5: Strong Classifier

After the weak classifiers, comes the strong classifier. Whatever values the weak classifiers give, we calculate the **weighted sum** of all these HAAR features and get the final value. This value is compared with a threshold:

- If greater than threshold → **object is detected**.
- Otherwise → object is not detected.

3 Summary of the HAAR Algorithm

HAAR is an algorithm used to detect faces or objects in an image.

1. Convert RGB image to grayscale, because HAAR deals with intensity values, not color.
2. Compute the integral image — for each pixel, calculate the cumulative sum of pixels lying above, left, and diagonal to it.
3. Divide the image into small windows of the desired size.
4. In each window, use the HAAR feature (black and white portion, like filters in deep learning).
5. Calculate the sum of pixels under the white portion and the sum of pixels under the black portion, and subtract them to get the HAAR feature, which is compared with a threshold (weak classifier output).
6. Use an algorithm like AdaBoost which does a weighted sum of all HAAR features and gives the final value, which is compared with a threshold. If greater, the object is there; otherwise, it is not.
7. We use multiple stages in HAAR on each window of an image, and each stage has multiple weak classifiers. A window that passes a stage goes to the next; otherwise, it is rejected.

4 Python Implementation of HAAR

4.1 Libraries Required

- **cv2**: used to deal with images or videos and perform operations on them.
- **NumPy**: used when dealing with arrays.
- **matplotlib.pyplot**: used to plot images and results.

Importing Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Listing 1: Libraries for HAAR implementation

4.2 Loading the XML Cascade Files

Download the provided HAAR cascade files, upload them to Google Drive, and mount the Drive in Google Colab. Copy the path of `haarcascade_frontalface_default.xml` and `haarcascade_eye.xml` into the code.

> **Role of XML Files.** The XML files contain trained data for HAAR Cascade, which is used to detect faces. They contain all the data about what kind of stage classifiers are used in HAAR. The `face_cascade` becomes the object used to find faces in an image; another object is used to find eyes inside detected faces.

The `CascadeClassifier` imported from cv2 uses the data from the XML file to train the HAAR cascade classifier. Once loaded, it is able to detect faces, and eyes in detected faces.

Loading Cascade Classifiers

```
face_cascade = cv2.CascadeClassifier(
    "/content/drive/MyDrive/class/haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    "/content/drive/MyDrive/class/haarcascade_eye.xml")
```

Listing 2: Building face and eye cascade objects

5 Structure of a HAAR Cascade XML File

5.1 Top-Level Tags

The `<opencv_storage>` opening and closing tags denote the file that contains information for HAAR. The classifier used is **AdaBoost (EDA boost)** and the features used are **HAAR features**. Height and width define the sub-window size. A classifier may use, for example, six weak classifiers, with a stage threshold such as -1.45 (i.e., the final threshold for that stage is 1.45).

5.2 Weak Classifier Structure

For the first weak classifier, a threshold such as 1.2 is used, and `haarFeatures` is 0 because HAAR uses intensity difference — it doesn't use any kind of categorical features.

> **Leaf logic in a weak classifier:**
>
> - If the HAAR feature value **is greater than** the threshold → go to the **left leaf** (choose the first value).
> - If the condition is **not true** → go to the other value (right leaf).

This is the case of the first weak classifier for the first stage. A similar format applies for the rest of the weak classifiers in stage 1, with different thresholds.

5.3 Number of Weak Classifiers per Stage

The first stage has six weak classifiers. The second stage has twelve weak classifiers. Each stage has two thresholds: one for the weak classifier and one for the strong classifier. The third stage has nine weak classifiers, the fourth has sixteen, the fifth twenty-three, the sixth twenty-seven.

Stage	Weak Classifiers
1	6
2	12
3	9
4	16
5	23
6	27
7	28
8	36
9	47
10	48

This pattern continues through stages up to stage 24. This code shows how to build different stages of HAAR; the number of weak classifiers in each stage can be chosen.

5.4 Defining HAAR Features in XML

After stages and weak classifiers, the next part defines the HAAR features themselves — the black and white rectangular filters that extract features.

Feature Rectangles. Each HAAR feature is defined by rectangles. For a rectangle, the XML specifies x and y coordinates, width, and height.

- **White rectangle**: e.g., $(x, y) = (0, 8)$, width–height $(20, 12)$, weightage -1.
- **Black rectangle**: e.g., $(x, y) = (0, 14)$, width–height $(2, 6)$, weightage $+2$.

The weightage means that once the sum of white pixels is computed, it is multiplied by -1; once the sum of black pixels is computed, it is multiplied by $+2$.

For example, if the sum of white pixels is 16, multiply by -1; if the sum of black pixels is 15, multiply by $+2$. This way, different types of HAAR features can be defined — edge, line, and four-rectangle features.

XML is a Blueprint. This XML file is already available and developers are using it directly. If you want to play around with things, you can build your own, but the pre-built XML can be used as-is. The eye and face XMLs have similar structures, with the only difference being the number of stages and the number of weak classifiers inside each stage.

6 Face and Eye Detection Functions

6.1 Detecting Faces

The `face_cascade` object is used to locate a face in an image using `detectMultiScale`. It knows the structure of the face-finding algorithm from the XML and runs it on the image to detect the face. A rectangle is drawn around the face with a chosen color and line thickness.

Detect Faces Function

```
def detect_faces(img):
    faces = face_cascade.detectMultiScale(img, 1.2, 5)
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 255, 255), 10)
```

```
    return img
```

Listing 3: Face detection using HAAR cascade

Parameters: **scaleFactor** is 1.2, **minNeighbors** is 5, rectangle color is white $(255, 255, 255)$, and line weight is 10.

6.2 Detecting Eyes

The `eye_cascade` is an object that knows the type of algorithm for detecting eyes (from `haarcascade_eye.xml`). It runs on the detected face to find the eyes, then draws a rectangle around them.

Detect Eyes Function

```
def detect_eyes(img):
    eyes = eye_cascade.detectMultiScale(img)
    for (x, y, w, h) in eyes:
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 255, 255), 10)
    return img
```

Listing 4: Eye detection inside detected faces

6.3 Reading and Copying the Image

We read the image (such as `close_up.jpg`), copy its path, and make multiple copies so that we can pass one copy to `detect_faces` and another copy to `detect_eyes`. We display the image with `matplotlib`. Three copies of the image are used: one for face detection and one for eye detection.

Running Face and Eye Detection

```
img = cv2.imread("/content/drive/MyDrive/class/close_up.jpg")
img_copy1 = img.copy()
img_copy2 = img.copy()

faces_img = detect_faces(img_copy1)
eyes_img = detect_eyes(img_copy2)
```

Listing 5: Running detection on image copies

7 Combined Face and Eye Detection

To locate both eyes and faces in an image, read the image file (cv2 reads it in BGR format), convert it to grayscale using `cv2.cvtColor` (since `cvtColor` accepts BGR and can convert to grayscale). Load the HAAR classifiers for face and eye, then detect faces.

For each detected face, a rectangle is drawn around the face in **blue**, and rectangles are drawn around eyes in **green**.

Face and Eye Detection Together

```
img = cv2.imread("/content/drive/MyDrive/class/image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = face_cascade.detectMultiScale(gray, 1.2, 5)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)
    roi_gray = gray[y:y + h, x:x + w]
    roi_color = img[y:y + h, x:x + w]
    eyes = eye_cascade.detectMultiScale(roi_gray)
```

```
    for (ex, ey, ew, eh) in eyes:
        cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh),
                      (0, 255, 0), 2)
```

Listing 6: Drawing blue around face, green around eyes

Color Encoding. $(255, 0, 0)$ indicates **blue** (OpenCV uses BGR), and the line thickness indicates how thick this line will be. In the code above, the thickness is 2.

8 Detecting Multiple Faces

We can have multiple faces in an image. Again we use `face_cascade` on the grayscale image. The `detectMultiScale` method is used on the grayscale image `gray`.

- An image with two faces → HAAR detects two faces with green bounding boxes.
- An image with four faces → HAAR detects all four faces with green bounding boxes.

Again, the `CascadeClassifier` reads the XML file and builds the classifier. XML is the blueprint; it is read and an object is built. This object knows the structure of the HAAR classifier and, using `detectMultiScale` on the grayscale image, finds the faces.

9 Non-Maximum Suppression for Overlapping Boxes

Sometimes images can have four or more faces, and HAAR can produce **multiple overlapping bounding boxes** — because HAAR detects the object at different stages. In one window it is possible to have overlapping bounding boxes, similar to YOLO.

Non-Maximum Suppression. Non-maximum suppression is a technique used to avoid overlap between bounding boxes when a detector produces multiple boxes for the same object. Only the correct bounding box, where the object is properly located, is kept.

This is especially useful for images with many faces (for example, five faces), where overlapping detections can occur. After applying non-maximum suppression, only the true bounding boxes remain.

Path Issues. When errors occur in HAAR detection, the most common cause is an incorrect path given to the XML files or images. The `CascadeClassifier` cannot find the XML, so `detectMultiScale` cannot run. Always check that the correct path is provided to XML files and images.

10 Introduction to HOG

HOG. HOG (Histogram of Oriented Gradients) is a feature extraction technique in computer vision used for object detection, such as detecting humans or faces. The main idea is that HOG captures the **shape** of the object using **edge directions**, because objects are better defined by their edges and structure, not by color.

- The human body has vertical edges.
- Faces have curved edges.

With these properties of edges, we are able to locate the objects; color is not the main component. HOG captures these patterns using **gradient orientation**.

11 Steps Involved in HOG

11.1 Step 1: Compute Gradients

For each pixel, calculate the **magnitude** and the **orientation** (edge direction).

11.2 Step 2: Divide Image Into Cells

Divide the image into small cells.

11.3 Step 3: Histogram of Gradient Directions

For each cell, create a **histogram of gradient directions**, which tells us which edge direction dominates in that cell.

> **Edges Define Shape.** Different edges and lines lead to shapes and parts of the object. Edges and structures are the main component.

11.4 Step 4: Block Normalization

After creating the histogram of gradients for each cell, do **normalization**: group these cells into **blocks** and normalize the values.

11.5 Step 5: Combine Histograms

Combine all the histograms and get the final **HOG feature descriptor**. This descriptor is passed to a classifier such as **MLP** or **SVM** so we can classify whether the region contains the object or not.

12 Why Use HOG and Its Limitations

12.1 Advantages

- Captures shape information.
- Used under different variations of light.
- Used for human detection.

12.2 Limitations

- Does not work well for complex backgrounds.
- Does not work well for large pose variations.

HOG is a classical feature extraction algorithm that uses shape and structure using edges to find the object in an image. It detects objects based on their shape. The main crux is that it uses gradient directions to calculate the object shape.

13 Python Implementation of HOG

13.1 Libraries Required

- `skimage.color`: handles color conversion (such as RGB to gray).
- `skimage.feature.hog`: handles feature extraction using HOG.
- `skimage.data` and contrast-related utilities: to import datasets and work with contrast.
- `matplotlib.pyplot`: for plotting.

HOG Imports

```
from skimage import color, data, exposure
from skimage.feature import hog
```

```
import matplotlib.pyplot as plt
```

Listing 7: Imports for HOG pipeline

13.2 HOG on an Astronaut Image

From `data`, import the astronaut image and convert it to grayscale. Then apply HOG with **9 orientations**, **pixels per cell** 8×8, and a **block of 2 cells**. Compute the HOG features and the HOG image, then plot.

HOG on Astronaut

```
image = color.rgb2gray(data.astronaut())

fd, hog_image = hog(image,
                    orientations=9,
                    pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2),
                    visualize=True)

plt.imshow(hog_image, cmap="gray")
plt.show()
```

Listing 8: HOG feature extraction and visualization

The input image is shown, and the HOG output visualizes the locations of the object where edges have better contrast. HOG captures where the object is located based on the edges.

13.3 HOG on Other Images (e.g., Cup and Saucer)

The same HOG can be applied to other images. For example, on a cup-and-saucer image, HOG locates the cup and the saucer plate properly. The parameters `orientations`, `pixels_per_cell`, and `cells_per_block` can be varied. For example, using `cells_per_block=(1, 1)` instead of $(2, 2)$ changes how many detailed features are captured.

14 Varying HOG Parameters

There are two ways of calculating HOG. In the alternate approach, we use different **orientations** and change `pixels_per_cell` and `cells_per_block`.

- Changing the number of orientations (e.g., from 9 to a different value, such as higher orientations) produces **more detailed features**.
- Some information may be missing between blocks with certain settings.
- Playing around with the `orientations` parameter yields different visualizations of features.

> **HOG parameters that control detail:**
>
> `orientations`, `pixels_per_cell`, `cells_per_block`
>
> Increasing the number of orientations increases the level of detail in the extracted features.

15 Practical Notes on HAAR vs HOG

15.1 When to Use HAAR

HAAR is useful when dealing with face detection. In future work on face detection, HAAR is needed.

15.2 When to Use HOG

HOG is better when finding the edges of objects, because it uses different directions of the gradient. For example, detecting plate or dish boundaries with better edge visualization works with HOG.

15.3 Limitations of Classical Algorithms

Both HAAR and HOG came before deep learning. Although they are able to detect faces, sometimes at many places they do not work. This is because they are **not learning-based algorithms**. They work well where the intensity difference is clear; where it is not, they fail.

15.4 Combining with Deep Learning

The world now uses AI and deep learning, but these methods are not forgotten. They are still used in various scenarios where features are first extracted with HAAR, and then deep learning is used to carry out further tasks.

16 Parameter Tuning for Face Detection

When bounding boxes do not cover a region properly (for example, the chin of a face), it is not that bounding boxes are a separate issue — the case is that `scaleFactor`, `minNeighbors`, and minimum size must be tuned. You must find the optimal values and play around with them; the detection varies accordingly.

> **Chin Detection Example.** In certain cases, the intensity difference is not enough. For example, if the difference between chin and neck is not much, the chin portion is not covered. If the chin and neck have a larger intensity difference, the chin is covered.

17 Summary: Key Points

- **HAAR** uses simple rectangular black-and-white patterns to detect objects (faces, eyes) based on pixel intensity differences.
- HAAR works on integral images, uses multi-stage weak classifiers with AdaBoost, and culminates in a strong classifier.
- The XML cascade files are blueprints specifying stages, weak classifiers, thresholds, and HAAR feature rectangles.
- In Python, `cv2.CascadeClassifier` loads the XML and `detectMultiScale` runs detection; `cv2.rectangle` draws bounding boxes.
- Grayscale conversion reduces computational complexity from three channels to one while preserving structural patterns.
- **Non-maximum suppression** avoids overlapping bounding boxes when multiple detections occur.
- **HOG** captures shape via gradient orientation: compute gradients, divide into cells, build histograms, normalize across blocks, and combine.
- HOG is classical and limited in complex backgrounds and large pose variations, but effective for shape-based detection like humans or plates.
- HAAR and HOG are classical feature extractors that can be combined with deep learning for modern applications.